

1. Вариант 23 Огонь по движущимся танкам

На прямоугольном поле перемещаются танки, а одно или несколько орудий пытаются их поразить. Каждый танк занимает одну клетку и на каждом такте может остаться на месте либо перейти в соседнюю допустимую клетку. Орудие получает сведения о последнем известном положении цели и выбирает координаты выстрела по случайной или прогнозирующей стратегии (не обязательно в позицию танка, пытаясь предугадать его возможное перемещение). Выстрел имеет задержку между наведением и попаданием. После формирования выстрела танк может переместиться, поэтому снаряд поражает указанную клетку, а не идентификатор цели. В одной клетке могут действовать правила прямого попадания, близкого разрыва и промаха.

Требуется разработать консольное приложение, моделирующее огонь по движущимся целям.

Входные параметры

- размеры поля и препятствия;
- число и начальные позиции танков и орудий;
- стратегии движения;
- дальность наблюдения;
- задержка и точность выстрела;
- радиус поражения (на выбор: от одной клетки до некоторого ущерба в соседних);
- запас снарядов и прочность танков;
- предельное число тактов. Отображаемые события Программа отображает перемещение танка, наблюдение цели, выбор точки прицеливания, выстрел, фактическое положение цели в момент попадания, промах, повреждение и уничтожение.

Инварианты

- танк не выходит за границы и не проходит через препятствие;
- один выстрел расходует один снаряд;
- попадание определяется по состоянию поля в момент разрыва;
- уничтоженный танк больше не перемещается и становится препятствием;
- значение прочности не становится отрицательным без фиксации уничтожения.

Завершение

Моделирование завершается после уничтожения всех танков, исчерпания боеприпасов или достижения ограничения тактов, или по прерыванию пользователем, если задан режим неограниченного по времени выполнения. Каждое орудие и танк, а также координатор поля являются самостоятельными участниками модели.

2. Сценарий

Сущность	Программное представление	Файлы
Поле (координатор)	struct Field, field_*	field.h/.c
Танк	struct Tank, tank_*	tank.h/.c
Орудие	struct Oreshnik, gun_*	gun.h/.c
Снаряд	struct Projectile, proj_*	projectile.h/.c
Событие календаря	struct Event	event.h/.c
Календарь событий	EventQueue (по времени, приоритету, порядку)	event.h/.c
Координатор всей имитации	struct SimContext	sim.h/.c
Наблюдатель (журнал)	функции io_printf, log_line	io.h/.c, log.h/.c
Генератор случайных чисел	struct Rng, rng_*	rng.h/.c

Модельное время - целочисленное (long long tick). Обработка идёт по очереди событий: ближайшее по времени событие извлекается, время продвигается, событие обрабатывается, при необходимости порождаются новые события.

Порядок обработки внутри одного такта определяется вторичным ключом priority (числовое значение типа события):

1. EV_TANK_MOVE = 10 - перемещения танков;
2. EV_GUN_OBSERVE = 20 - наблюдения;
3. EV_GUN_FIRE = 30 - выстрелы;
4. EV_PROJECTILE_IMPACT = 40 - разрывы снарядов.

Это обеспечивает инвариант “попадание определяется по состоянию поля в момент разрыва” - разрывы обрабатываются после всех перемещений того же такта. Третичный ключ seq - гарантирует детерминизм при полном совпадении (time, priority).

3. Установленные правила

3.1. Поле и перемещение

- Поле прямоугольное, задаётся $w \times h$, координаты клеток - (x, y) , $x \in [0, w-1]$, $y \in [0, h-1]$.

- Каждая клетка имеет одно из трёх состояний: пусто (CELL_EMPTY), препятствие (CELL_OBSTACLE), обломки (CELL_WRECK).
- Препятствия задаются из конфигурации; обломки появляются при уничтожении танка.
- Шаг танка - на одну клетку по одной из четырёх сторон (N, E, S, W) либо “стоять”. Диагоналей нет.
- Танк может шагнуть в клетку, если она в границах поля, не препятствие, не обломки и не занята другим живым танком.
- Танки блокируют друг друга: два танка никогда не окажутся в одной клетке.
- Живой танк не блокирует движение орудия (орудия стоят вне игрового процесса перемещений), но занимает свою клетку.

3.2. Стратегии движения танка

| Стратегия | Правило |

| stay | Танк всегда остаётся на месте. |

| random_walk | На каждом такте равновероятно выбирается одно из 5 действий (4 шага + “стоять”), но только среди допустимых; если все шаги запрещены, танк стоит. |

| straight | Танк идёт в текущем направлении, пока не упрётся (в границу, препятствие, обломки или другой танк). После упора выбирает направление (влево/вправо равновероятно). |

| patrol | Танк ходит между двумя точками А и В: сначала сокращает $|dx|$, затем $|dy|$. При достижении точки маршрут переключается. Если прямое движение заблокировано - пробует вторую ось. Если и она закрыта - стоит. |

Движение планируется на каждый такт: обработчик EV_TANK_MOVE по завершении всегда перепланирует следующее событие на $t+1$, если танк жив.

3.3. Наблюдение орудия

- Дальность наблюдения - по манхэттенскому расстоянию: $d = |x_{gun} - x_{tank}| + |y_{gun} - y_{tank}|$.

- Орудие на каждом такте (событие EV_GUN_OBSERVE) ищет всех живых танков в радиусе range и запоминает ближайшего как текущую цель.

- Для цели хранятся две точки: last (последняя виденная) и prev (виденная до неё). prev нужна для стратегии прогнозирования.

- Если ни одного танка в радиусе нет, цель считается потерянной, $has_target = 0$. Орудие не стреляет, пока не восстановит цель.

- Наблюдение повторяется каждый такт - орудие работает с самой актуальной на текущий момент информацией о цели.

3.4. Выбор точки прицеливания

Орудие (событие EV_GUN_FIRE) выбирает точку по одной из трёх стратегий:

| Стратегия | Формула |

| last | $(last_x, last_y)$ |

| predict | $(last_x + (last_x - prev_x), last_y + (last_y - prev_y))$ |

| random | $(last_x \pm 2, last_y \pm 2)$ - случайное смещение вокруг последней позиции |

Далее применяется имитация неточности: с вероятностью $(100 - accuracy)\%$ к прицелу добавляется случайное смещение $(\pm 1, \pm 1)$. Точка зажимается в границы поля.

Снаряд летит в клетку, а не в идентификатор танка.

3.5. Выстрел и снаряд

- Выстрел возможен только если у орудия "ammo > 0" и has_target = 1.

- Каждый выстрел списывает один снаряд: ammo--.

- Перезарядка: следующая попытка выстрела планируется на $t + reload$, независимо от того, был ли произведён выстрел (таким образом, «пустая» попытка тоже расходует темп перезарядки).

- Каждому снаряду назначается время разрыва $impact_time = t + shot_delay$. Снаряд учитывается в массиве projectiles и снимается с учёта после разрыва.

3.6. Разрыв снаряда

Для каждого живого танка вычисляется манхэттенское расстояние до клетки разрыва:

| Условие | Класс | Урон |

| $d == 0$ | прямое попадание | damage_direct |

| $0 < d \leq blast_radius$ | близкий разрыв | damage_splash |

| $d > blast_radius$ | промах | |

Один разрыв может поразить несколько танков одновременно (например, при большом радиусе). Если поражён хотя бы один танк выстрел засчитывается как попадание, иначе как промах.

3.7. Урон и уничтожение танка

- hp уменьшается на величину урона.

- Инвариант "hp" не становится отрицательным без фиксации уничтожения: перед присваиванием проверяется, что $hp - dmg \leq 0$; в этом случае "hp" принудительно устанавливается в 0, и фиксируется факт уничтожения.

- Уничтоженный танк:

- перестаёт двигаться (alive = 0, следующее событие EV_TANK_MOVE не планируется);
- становится препятствием: клетка помечается CELL_WRECK, другие танки через неё проходить не могут;
- исключается из всех проверок наблюдения и попадания.

3.8. Условия завершения

Моделирование завершается при первом из событий:

1. все танки уничтожены - REASON_ALL_DESTROYED;
2. у всех орудий ammo == 0 и в воздухе нет снарядов - REASON_NO_AMMO;
3. tick ≥ max_ticks (если max_ticks > 0) - REASON_MAX_TICKS;
4. поступил SIGINT или SIGTERM → REASON_INTERRUPTED.

По любому из вариантов программа печатает причину завершения, итоговую статистику и корректно освобождает ресурсы.

4. Пользовательский интерфейс

4.1. Способы запуска

Программа поддерживает три источника параметров, применяемых по следующему приоритету:

1. Аргументы командной строки(высший приоритет);
2. Файл конфигурации (--config <file>);
3. Интерактивный диалог (--interactive);
4. Значения по умолчанию - если ничего из выше не задано, запускается демонстрационный сценарий 12×8 с двумя танками и одним орудием.

4.2. Аргументы командной строки

Аргумент	Значение	По умолчанию
--seed N	начальное значение ГСЧ; 0 - взять из системного времени	42
--delay MS	задержка между тактами, мс	100
--max-ticks N	предел тактов; 0 - без предела	200
--config FILE	прочитать параметры из файла	
--log FILE	зеркалировать журнал в файл	
--no-field	не печатать поле каждый такт	поле печатается
--interactive	задать параметры через диалог	

| --help | вывести справку | |

4.3. Файл конфигурации

Формат - ключ значение

Файл перебивает значения по умолчанию, но ****уступает**** явным аргументам командной строки.

4.4. Интерактивный диалог

Запускается флагом --interactive. Каждый вопрос допускает пустой ответ - тогда принимается значение по умолчанию (оно показано в [...]). Ввод валидируется, при ошибке запрос повторяется с пояснением.

Порядок ввода:

1. Seed - если 0, берётся текущее системное время.
2. Поле: ширина и высота.
3. Препятствия: количество, затем либо автоматическое размещение с проверкой занятости, либо ручной ввод координат каждой клетки.
4. Танки: количество, для каждого - позиция (в допустимых границах, вне препятствий и других танков), hp, стратегия. Для straight - начальное направление, для patrol - две точки маршрута с проверкой проходимости.
5. Орудия: количество, для каждого - позиция, боезапас, дальность наблюдения, время перезарядки и задержки в полете, точность прицела, радиус поражения, урон в клетке попадания, урон в радиусе, стратегия.
6. Параметры симуляции: задержка между тактами, предел тактов, флаг печати поля.

По завершении ввода печатается сводка конфигурации - пользователь видит, что именно будет запущено.

5. Расшифровка ввода

5.1. [ГСЧ]

- Seed (0 = случайный) - начальное состояние генератора. Одинаковый seed → идентичный прогон (воспроизводимость).

5.2. [Поле]

- Ширина - количество клеток по оси X (мин. 2, макс. 200).

- Высота - количество клеток по оси Y (мин. 2, макс. 200).

5.3. [Препятствия]

- Сколько препятствий? - сколько статичных клеток закрыть. Диапазон $0..w \cdot h - 1$.
- Разместить случайно? - y - координаты выбираются ГСЧ с проверкой на занятость; n - пользователь вводит каждую клетку вручную.

5.4. [Танки]

- Сколько танков? - $0..32$.
- Позиция $x y$ - начальные координаты; проверяется, что клетка проходима и не занята другим танком.
- Прочность (hp) - начальное здоровье ($1..100000$). Урон вычитается из него; при $hp \leq 0$ танк уничтожается.
- Стратегия движения - одна из четырёх (3.2).
- Для *straight*: Начальное направление - вверх/вправо/вниз/влево.
- Для *patrol*: точка А и точка В - две клетки, между которыми ходит танк; обе должны быть проходимы.

5.5. [Орудия]

- Сколько орудий? - $0..16$.
- Позиция $x y$ - начальные координаты; клетка должна быть свободна.
- Боезапас - сколько всего выстрелов может сделать орудие ($0..100000$). Каждый выстрел списывает единицу.
- Дальность наблюдения - максимальное манхэттенское расстояние до танка, при котором орудие его видит.
- Перезарядка (тактов) - сколько тактов проходит между двумя попытками выстрела.
- Задержка полёта снаряда (тактов) - сколько тактов снаряд летит до разрыва.
- Точность прицела ($0..100$) - вероятность, что прицел не будет искажён; при 100 - идеальное наведение, при 0 всегда смещение ± 1 .
- Радиус поражения - максимальное манхэттенское расстояние от точки разрыва, в котором танк получает урон. 0 - только клетка попадания.
- Урон в клетке попадания - урон при прямом попадании ($d = 0$).
- Урон в радиусе - урон при близком разрыве ($0 < d \leq \text{blast_radius}$).
- Стратегия прицеливания - *random*, *last*, *predict*.

5.6. [Параметры симуляции]

- Задержка между тактами, мс - пауза *nanosleep* между тактами (0 - без задержки, для тестов).

- Предел тактов (0 = без предела) - сколько тактов максимум моделировать.
- Печатать состояние поля каждый такт? - если [y], перед каждым тактом будет напечатано поле.

6. Режимы и конфигурации для пользователя

6.1. Режимы запуска

- | Сценарий | Команда |
- | Демонстрационный сценарий | `./tank_sim` |
- | С фиксированным seed и задержкой | `./tank_sim --seed S --delay D` |
- | Ограниченный по тактам | `./tank_sim --max-ticks T` |
- | Полностью интерактивный ввод | `./tank_sim --interactive` |
- | Из файла конфигурации | `./tank_sim --config tests/basic.cfg` |
- | С записью в лог | `./tank_sim --log tank_sim.log` |
- | Без печати поля | `./tank_sim --no-field` |
- | Справка | `./tank_sim --help` |

6.2. Комбинирование

Все флаги комбинируются, кроме `--interactive` и `--config` - при одновременном использовании интерактив запускается поверх значений из файла. Явные аргументы `--seed/--delay/--max-ticks` перебивают соответствующие ключи из конфиг-файла.

6.3. Прерывание

- Ctrl+C в терминале посылает SIGINT;
- kill посылает SIGTERM;
- оба обрабатываются одинаково: главный цикл завершается на ближайшей итерации, печатается статистика, лог закрывается через `fsync` и `close`.

7. Реализация пользовательского интерфейса (техническая часть)

Ввод - через системный вызов `read(STDIN_FILENO, ...)` по одному байту; такое чтение устойчиво к пайпам и к длинным строкам, идущим через `echo` или перенаправление.

Вывод - через `write(STDOUT_FILENO, ...)`; форматирование строки выполняется `vsnprintf` в стековый буфер, а сам вывод - системным вызовом (ввод-вывод через дескрипторы).

Лог - открывается через `open(O_WRONLY | O_CREAT | O_TRUNC)`. Дублирование вывода достигается функцией `io_set_mirror(fd)`: каждый вызов `io_printf` дополнительно пишет ту же строку в лог-дескриптор. При завершении лог синхронизируется через `fsync` и закрывается через `close`.

Задержки - nanosleep, что корректно реагирует на сигналы (без SA_RESTART): Ctrl+C прерывает паузу, и цикл проверяет флаг.

Обработчики сигналов - устанавливаются через sigaction на SIGINT и SIGTERM. Обработчик только выставляет volatile sig_atomic_t.

8. Дополнительно

1. Doxygen комментарии (можно собрать через make doc).
2. Лог по завершении (файл открывается через --log, зеркалирует stdout через io_set_mirror).
3. Тестовые наборы - папка tests/ со скриптом run_all.sh (можно запустить через make test).
4. Возможность работы через конфиг-файл.
5. Воспроизводимый seed - ГСЧ на rand_r со своим состоянием, не зависящий от глобального.
6. Регулируемая задержка отображения --delay и интерактивный параметр.
7. Итоговая статистика.
8. Makefile

9. Сборка и запуск

make clean

make

./tank_sim (с нужным аргументом)